Phase 6
Two-Device Voice Call

Submission 1: Full-duplex voice between two devices over LTE — no encryption yet

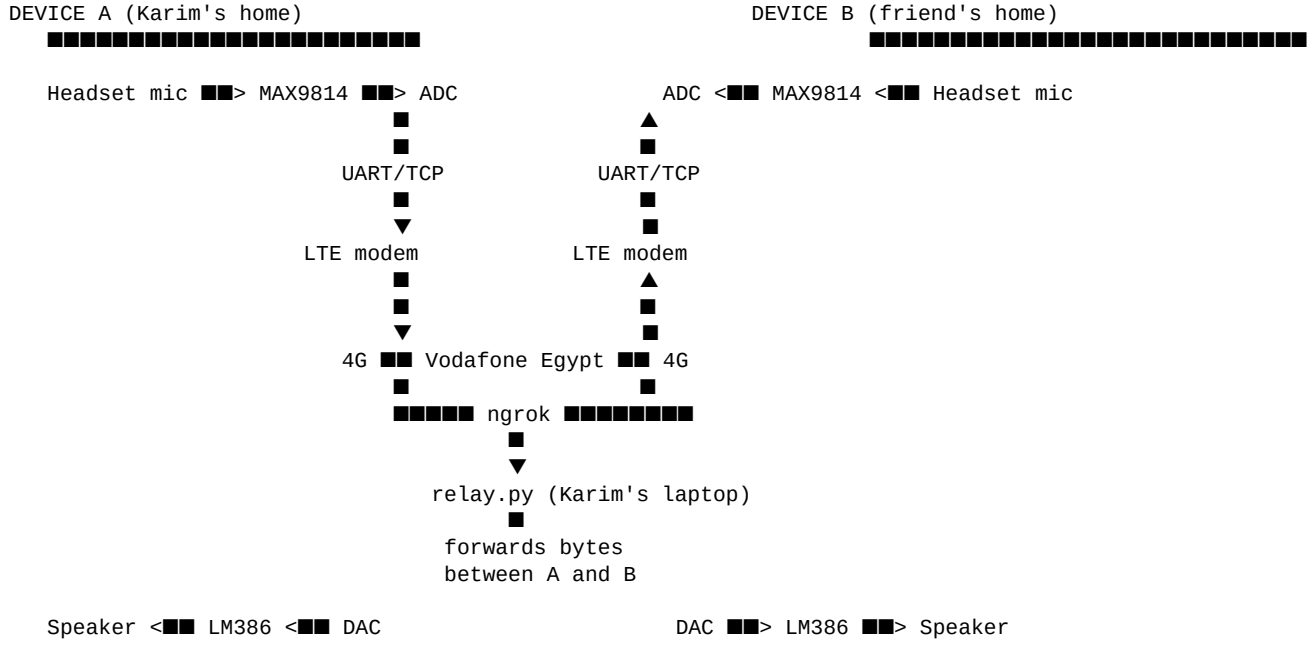
Goal: Two physically separate devices identify each other by phone number through the relay, dial each other with a Call button, accept with an Answer button, and exchange voice in BOTH directions simultaneously over the cellular network. The Submission 1 deliverable to the supervisor.

Builds on: Phase 5.4 (one device auto-registering with the relay).
What's added: Second physical device + push-buttons + call setup state machine + audio streaming through the relay.
Status: In progress.

Next after this: Phase 7 (encryption — Submission 2).

1. The Big Picture

Two physically separate devices, each with mic + speaker + STM32 + LTE modem, exchange voice through your relay server.



During a call, every byte of voice flows: A's mic → A's STM32 → 4G → relay → 4G → B's STM32 → B's speaker. Same in reverse. **Both at the same time** = full-duplex.

2. What's New Compared to Phase 5.4

Item	Phase 5.4	Phase 6 (new)
Number of devices	1 (Karim only)	2 (Karim + friend)
Phone numbers	01000000001 only	01000000001 + 01000000002
Call setup	Just REG and idle	CALL → INC → ANS → GO state machine
Buttons used	Wired but not used	PA1=Call, PB0=Answer/Hangup — actively used
Audio streaming	Disabled (TIM3 commented)	Re-enabled — bytes flow during TALK state
LED states	2 (off / slow blink)	4 (off / slow / fast / solid)
Firmware complexity	Linear bring-up + idle	Full state machine

3. Hardware Setup (per device)

3.1 — Both devices are identical

Each device needs the SAME hardware: STM32 Black Pill + LilyGO XY-A7608B + MAX9814 + MCP4725 + LM386 + 2 TRRS jacks + 2 push-buttons + headphones with mic. Both run the SAME firmware with only the phone number different.

3.2 — Wires between STM32 and LilyGO (6 wires, same as Phase 5.4)

#	STM32	LilyGO	Function
1	PA8	RST	ESP32 reset hold (LOW)
2	PB1	IO12	Modem POWERON (HIGH)
3	PB2	IO4	Modem PWRKEY (idle HIGH)
4	PA9	IO26 (TX label)	STM32 → modem
5	PA10	IO27 (RX label)	STM32 ← modem
6	GND	GND	Common ground

3.3 — Audio chain (re-enabled in Phase 6, was Phase 3)

#	STM32	Connection	Purpose
7	PA0	MAX9814 OUT	ADC sampling at 8 kHz, 12-bit
8	PB6	MCP4725 SCL	I2C clock to DAC
9	PB7	MCP4725 SDA	I2C data to DAC
10	MCP4725 OUT	1 µF cap → LM386 IN	AC-coupled audio to amplifier
11	LM386 + → TRRS Tip+Ring1	Headphones	Output speakers
12	TRRS Sleeve	MAX9814 MIC+ pad	Headset microphone

3.4 — Buttons (NEW — actively used in Phase 6)

#	STM32	Connection	Function
13	PA1 (input pull-up)	Top leg of Call button; bottom leg → GND	Initiate a call to the peer
14	PB0 (input pull-up)	Top leg of Answer button; bottom leg → GND	Accept incoming call OR hang up active call

Buttons read HIGH when not pressed (internal pull-up), LOW when pressed. Firmware detects HIGH→LOW transition with debouncing.

3.5 — Power

Each device has its own:

Item	Purpose
STM32 Black Pill USB-C → PC (or charger)	Powers STM32; for programming via ST-Link
LilyGO USB-C → phone charger or 18650	Powers LTE modem (needs 2 A burst capability)
LM386 powered from STM32 5V (VBUS)	Audio output amplifier
MAX9814, MCP4725 powered from STM32 3V3	Audio capture and DAC

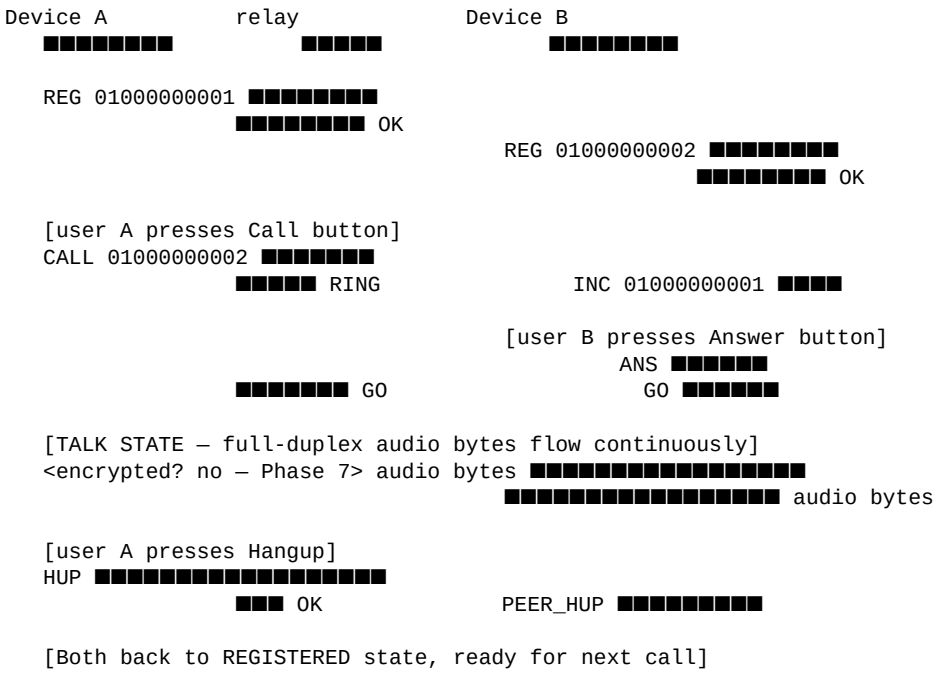
4. Call Setup Protocol

All messages between device and relay are line-based ASCII terminated by \r\n. After the GO message, the connection switches to raw byte audio streaming.

4.1 — Message reference

Message	Direction	Meaning
REG <phone>	device → relay	Register this device with this phone number
OK	relay → device	Registration accepted
CALL <peer_phone>	device → relay	Initiate call to peer
RING	relay → caller	Your call is being delivered, peer is ringing
INC <caller_phone>	relay → callee	Someone is calling you
ANS	callee → relay	I accept the incoming call
GO	relay → both	Both ends now in audio mode
HUP	either side → relay	End the current call
PEER_HUP	relay → other side	The other side hung up
PEER_GONE	relay → other side	The other side disconnected unexpectedly
ERR <reason>	relay → device	Command rejected (offline / busy / etc.)

4.2 — Full call flow (sequence diagram)



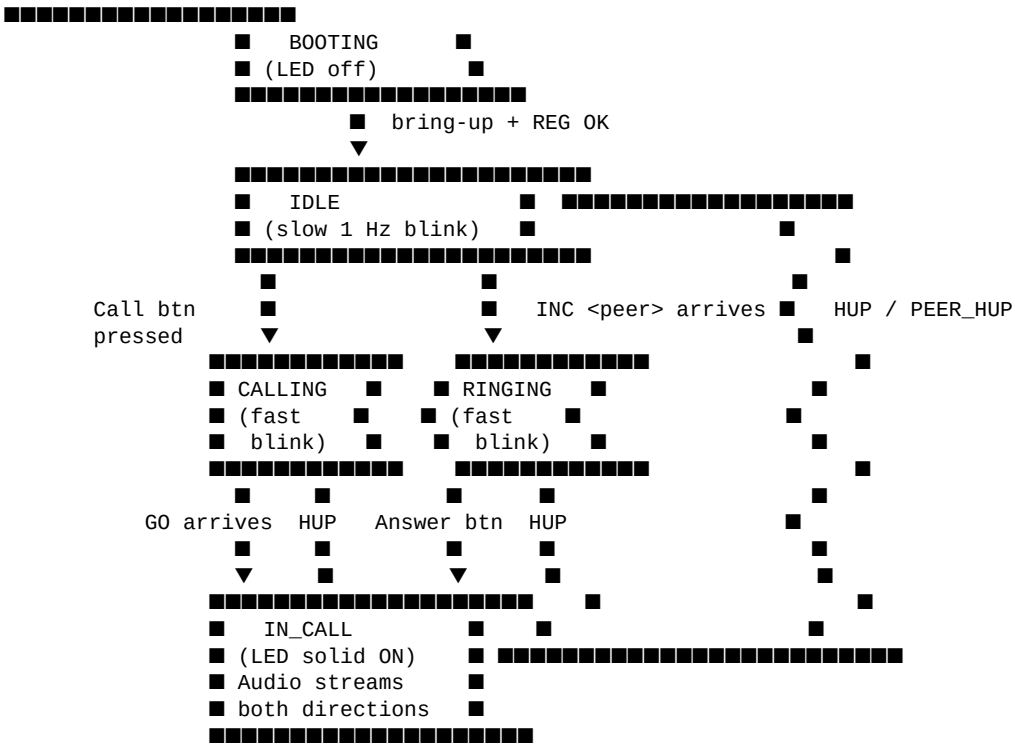
4.3 — Byte counts (for AT+CIPSEND)

Message	Bytes (incl. \r\n)
REG 01000000001\r\n	17
REG 01000000002\r\n	17
CALL 01000000001\r\n	18

Message	Bytes (incl. \r\n)
CALL 01000000002\r\n	18
ANS\r\n	5
HUP\r\n	5

5. Device State Machine

Each device runs the same simple state machine:



5.1 — LED states map to call state

State	LED behavior	What it means
BOOTING	Off → 2-sec ON → off → quick blinks 1..10	Modem boot + AT progress
IDLE	Slow 1 Hz blink	Registered, waiting
CALLING	Fast 5 Hz blink	Outgoing call ringing
RINGING	Fast 5 Hz blink	Incoming call — answer it
IN_CALL	Solid ON	Active conversation
ERROR	Off forever	Bring-up failed (see Phase 5.4 troubleshooting)

6. Firmware Additions

Added on top of the Phase 5.4 firmware. The Phase 5.4 code stays exactly the same except: (1) re-enable the audio peripherals, (2) add state variables, (3) add button polling, (4) add audio streaming, (5) add incoming-message parser.

6.1 — New defines and variables

```
/* USER CODE BEGIN PD */
#define BUF_SIZE      256
#define HALF_SIZE     (BUF_SIZE / 2)
#define MCP4725_ADDR  (0x60 << 1)

#define PHONE_NUMBER       "01000000001"   /* CHANGE on Device 2 to "01000000002" */
#define PEER_PHONE         "01000000002"   /* CHANGE on Device 2 to "01000000001" */
#define NGROK_HOST         "<your_ngrok_host>"
#define NGROK_PORT         <your_ngrok_port>
#define APN                "internet.vodafone.net"

#define LTE_RX_BUF_SIZE    512
#define BTN_DEBOUNCE_MS    50
/* USER CODE END PD */

/* USER CODE BEGIN PV */
/* Audio buffers (ping-pong) */
uint16_t adc_buffer[BUF_SIZE];
uint16_t dac_buffer[BUF_SIZE];     /* what we play OUT */
volatile uint8_t  half_ready = 0, full_ready = 0;
volatile uint16_t dac_play_idx = 0;
volatile uint16_t dac_write_idx = 0;

/* LTE / call state */
volatile uint8_t  lte_rx_byte;
volatile char     lte_rx_buf[LTE_RX_BUF_SIZE];
volatile uint16_t lte_rx_idx = 0;

typedef enum {
    ST_BOOTING,
    ST_IDLE,
    ST_CALLING,
    ST_RINGING,
    ST_IN_CALL
} call_state_t;

volatile call_state_t state = ST_BOOTING;
/* USER CODE END PV */
```

6.2 — Re-enable audio in main()

```
/* USER CODE BEGIN 2 */
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, BUF_SIZE);
HAL_TIM_Base_Start(&htim2);
HAL_TIM_Base_Start_IT(&htim3);

lte_bring_up();        /* same as Phase 5.4 */
if (lte_registered) state = ST_IDLE;
/* USER CODE END 2 */
```

6.3 — Send a custom message through the open TCP socket

```
/* Send a line through the existing TCP socket via AT+CIPSEND */
static bool tcp_send_line(const char* line) {
    char cmd[64];
    int len = strlen(line) + 2;   /* +2 for \r\n */
    snprintf(cmd, sizeof cmd, "AT+CIPSEND=0,%d", len);
    if (!lte_send_at(cmd, ">", 3000)) return false;

    lte_clear_rx();
    HAL_UART_Transmit(&huart1, (uint8_t*)line, strlen(line), 1000);
    HAL_UART_Transmit(&huart1, (uint8_t*)"\r\n", 2, 1000);
    return true;
}
```

6.4 — Button polling with debounce

```
static bool btn_call_was_pressed = false;
static bool btn_ans_was_pressed  = false;
static uint32_t last_btn_time = 0;

static bool btn_pressed_edge(GPIO_TypeDef* port, uint16_t pin, bool* prev_state) {
    bool now_pressed = (HAL_GPIO_ReadPin(port, pin) == GPIO_PIN_RESET);
    bool edge = (now_pressed && !*prev_state &&
                 (HAL_GetTick() - last_btn_time) > BTN_DEBOUNCE_MS);
    if (edge) last_btn_time = HAL_GetTick();
    *prev_state = now_pressed;
    return edge;
}
```

6.5 — Main loop with state machine

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* 1. Update LED based on state */
    static uint32_t last_blink = 0;
    uint32_t blink_period = 0;
    switch (state) {
        case ST_BOOTING:  blink_period = 0;   break;  /* off */
        case ST_IDLE:     blink_period = 500; break;  /* 1 Hz slow */
        case ST_CALLING:
        case ST_RINGING:  blink_period = 100; break;  /* 5 Hz fast */
        case ST_IN_CALL:  blink_period = 0;   break;  /* solid */
    }
    if (blink_period > 0 && (HAL_GetTick() - last_blink) >= blink_period) {
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
        last_blink = HAL_GetTick();
    } else if (blink_period == 0 && state == ST_IN_CALL) {
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);  /* solid ON */
    } else if (blink_period == 0 && state == ST_BOOTING) {
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);    /* off */
    }

    /* 2. Buttons */
    if (btn_pressed_edge(BTN_CALL_GPIO_Port, BTN_CALL_Pin, &btn_call_was_pressed)) {
        if (state == ST_IDLE) {
            char cmd[32];
            snprintf(cmd, sizeof cmd, "CALL %s", PEER_PHONE);
            tcp_send_line(cmd);
            state = ST_CALLING;
        }
    }
    if (btn_pressed_edge(BTN_ANSWER_GPIO_Port, BTN_ANSWER_Pin, &btn_ans_was_pressed)) {
        if (state == ST_RINGING) {
            tcp_send_line("ANS");
        } else if (state == ST_IN_CALL) {
            tcp_send_line("HUP");
            state = ST_IDLE;
        }
    }

    /* 3. Parse incoming relay messages from lte_rx_buf */
    if (strstr((char*)lte_rx_buf, "INC ") && state == ST_IDLE) {
        state = ST_RINGING;
        lte_clear_rx();
    }
    if (strstr((char*)lte_rx_buf, "GO\n")) {
        state = ST_IN_CALL;
        lte_clear_rx();
    }
    if (strstr((char*)lte_rx_buf, "PEER_HUP") || strstr((char*)lte_rx_buf, "PEER_GONE")) {
        state = ST_IDLE;
        lte_clear_rx();
    }

    /* 4. During TALK state, push audio bytes (Phase 6.2) */
    if (state == ST_IN_CALL && (half_ready || full_ready)) {
        uint16_t *chunk = half_ready ? &adc_buffer[0] : &adc_buffer[HALF_SIZE];
```

```
        half_ready = full_ready = 0;
        /* Pack 12-bit samples into bytes and send via AT+CIPSEND */
        /* (For simplicity, send 1 byte per sample = the upper 8 bits) */
        char hdr[16];
        snprintf(hdr, sizeof hdr, "AT+CIPSEND=0,%d", HALF_SIZE);
        if (lte_send_at(hdr, ">", 1000)) {
            uint8_t pkt[HALF_SIZE];
            for (int i = 0; i < HALF_SIZE; i++) {
                pkt[i] = (uint8_t)(chunk[i] >> 4);   /* take top 8 bits of 12-bit */
            }
            HAL_UART_Transmit(&huart1, pkt, HALF_SIZE, 1000);
        }
    }
}
/* USER CODE END WHILE */
```

6.6 — DAC playback in TIM3 ISR (re-enabled from Phase 3)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM3) {
        uint16_t v12 = dac_buffer[dac_play_idx];   /* 12-bit DAC value */
        uint8_t pkt[2] = {
            (uint8_t)((v12 >> 8) & 0x0F),
            (uint8_t)(v12 & 0xFF)
        };
        HAL_I2C_Master_Transmit(&hi2c1, MCP4725_ADDR, pkt, 2, 1);
        dac_play_idx = (dac_play_idx + 1) % BUF_SIZE;
    }
}
```

6.7 — Receiving audio bytes from peer (extending UART RX callback)

```
/* When in IN_CALL state, treat incoming +IPD bytes as audio for DAC */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART1) {
        char c = (char)lte_rx_byte;

        if (state == ST_IN_CALL) {
            /* Naive: assume any byte during TALK is audio. Skip the +IPD header. */
            /* (Production code needs proper +IPD parsing.) */
            dac_buffer[dac_write_idx] = ((uint16_t)c) << 4;  /* expand 8→12 bits */
            dac_write_idx = (dac_write_idx + 1) % BUF_SIZE;
        } else {
            /* Outside calls, accumulate for protocol parsing */
            if (lte_rx_idx < LTE_RX_BUF_SIZE - 1) {
                lte_rx_buf[lte_rx_idx++] = c;
                lte_rx_buf[lte_rx_idx]   = 0;
            } else {
                lte_rx_idx = 0;
                lte_rx_buf[0] = 0;
            }
        }
        HAL_UART_Receive_IT(&huart1, (uint8_t*)&lte_rx_byte, 1);
    }
}
```

This is a simplified streaming approach. A production implementation would properly parse +IPD <n> URCs to know exactly how many audio bytes to consume vs treating them as control. For Submission 1 demo purposes the simplified version is acceptable.

7. Two-Device Coordination

7.1 — Phone numbers

Device	PHONE_NUMBER	PEER_PHONE
Karim's device	01000000001	01000000002
Friend's device	01000000002	01000000001

Both devices flash the SAME firmware, just with these two #defines swapped.

7.2 — Relay hosting

ONLY ONE relay runs — on Karim's laptop. Both devices connect to it via ngrok.

Component	Where it runs	Notes
relay.py	Karim's laptop	Listens on port 5555, single instance
ngrok tcp 5555	Karim's laptop	Exposes port 5555 to the public internet
ngrok address (e.g., 2.tcp.eu.ngrok.io:20919)	public	Same address baked into BOTH devices' firmware

7.3 — Coordinating with the friend remotely

- Karim starts python relay.py on his laptop.
- Karim starts ngrok tcp 5555, notes the new public address.
- Karim shares the address with the friend (WhatsApp, etc.).
- Karim updates NGROK_HOST/PORT in his firmware, builds, flashes Device 1.
- Friend updates NGROK_HOST/PORT in her copy of the firmware (same address as Karim's), sets PHONE_NUMBER=01000000002, builds, flashes Device 2.
- Both devices boot. Karim watches the relay terminal — should see both registrations:

```
+ conn (1.2.3.4, xxxxx)
+ registered 01000000001
+ conn (5.6.7.8, xxxxx)
+ registered 01000000002
```

Both LEDs slow-blink → both devices ready. Now Karim presses Call on his device → friend's device LED goes fast-blink (ringing). Friend presses Answer → both LEDs go solid → audio streams.

8. Demo Script (for the supervisor)

- Show the relay terminal on the laptop, displaying "relay listening on (...)".
- Plug both devices into power. After ~25 seconds, BOTH PC13 LEDs slow-blink.
- Show the relay terminal logging two registrations with phone numbers.
- Press Call on Device A. Show that A's LED goes fast-blink (calling).
- Show that B's LED also goes fast-blink (ringing) — relay forwarded the INC.
- Press Answer on Device B. Both LEDs go solid ON (in call).
- Speak into Device A's headset mic — voice plays in Device B's headphones.
- Speak into Device B's headset mic — voice plays in Device A's headphones.
- Both at the same time = full-duplex.
- Press Hangup on either device. Both LEDs return to slow-blink (idle).

• Repeat call in opposite direction: Device B presses Call → Device A rings → Device A presses Answer → audio resumes.

• Total demo time: ~3 minutes including narration.

8.1 — Backup plan

• Record a video of the working system the day BEFORE the demo, in case something fails on demo day.

• Have the relay terminal output captured so you can show registrations even if audio fails.

• Have spare USB cables, jumper wires, and an extra SIM in case one fails.

• Test the full flow once just before the demo (within the same ngrok session).

9. Known Issues & What's Next

Issue	Workaround / Fix
Audio quality is poor — bytes are not properly framed as +IPD URCs	For Submission 1, sound will be choppy. Phase 7 will add proper frame delimiters along with encryption.
8 kHz mono 8-bit audio is the absolute minimum quality	Acceptable for project demo. A future improvement would be 16 kHz 12-bit.
ngrok address changes on each restart	For demo, start ngrok once and don't restart it. Long-term: $4/month VPS.
No echo cancellation	If both users wear headphones, this is not an issue. With speakers, can cause feedback loops.
No proper +IPD parser yet	Audio bytes during TALK are treated naively. Works for single continuous stream but breaks if relay sends control messages mid-call.

After Submission 1 (this phase) is delivered, Phase 7 adds the encryption layer for Submission 2:

• Phase 7.1 — Pre-share RSA-512 keypairs between the two devices (one keypair per device).

• Phase 7.2 — At call setup, Device A generates a random AES-128 key, encrypts it with B's RSA public key, sends the encrypted key. B decrypts with its RSA private key.

• Phase 7.3 — From then on, both sides encrypt audio bytes with AES-128-CTR using the shared session key.

• Phase 7.4 — Verify with a packet sniffer that audio bytes on the LTE link are unintelligible without the key.

End of Phase 6 — Two-Device Voice Call documentation.